

**A Differentiable Simulation Testbed for Longitudinal Train Dynamics:
Reference Simulator, Real-Terrain Route Pipeline, and Research Console for
a Graph Neural Surrogate**

George James*

School of Engineering, Colorado State University Pueblo
2200 Bonforte Blvd., Pueblo, CO 81001
Email: gl.james1@pack.csupueblo.edu

Trung Duong, Ph.D.

School of Engineering, Colorado State University Pueblo
2200 Bonforte Blvd., Pueblo, CO 81001
Email: trung.duong@csupueblo.edu

Md Islam, Ph.D.

School of Engineering, Colorado State University Pueblo
2200 Bonforte Blvd., Pueblo, CO 81001
Email: md.islam@csupueblo.edu

Saqib Gulzar, Ph.D.

School of Engineering, Colorado State University Pueblo
2200 Bonforte Blvd., Pueblo, CO 81001
Email: saqib.gulzar@csupueblo.edu

Total Number of Pages: 19

Submitted: July 28, 2026

*Corresponding author

1 ABSTRACT

2 High-fidelity simulation of longitudinal train dynamics (LTD) resolves nonlinear slack couplers,
3 per-vehicle grade and curvature, actuator lag, and power-limited traction, but its computational cost
4 places it beyond the repeated evaluation required for data-driven control design and operational
5 planning. A fast, differentiable surrogate would remove this barrier; training one, however, first
6 requires a resource that does not yet exist in the open, namely a reproducible, real-terrain LTD data
7 generator spanning a wide family of consists and corridors. This paper documents that testbed.
8 The coupled-vehicle system is formulated as a chain graph whose tensorized node and edge state
9 admits arbitrary consist length without padding, and the target surrogate is expressed as a physics-
10 residual correction to a known force-balance right-hand side so that it remains differentiable end
11 to end. Three components are documented against this formulation: a stage-verified reference
12 simulator, a pipeline that converts OpenStreetMap track geometry and USGS elevation data into
13 the fields the simulator consumes, and TrainSet, a desktop console that renders consists, routes,
14 and control profiles as named, serializable artifacts shared by interactive and batch use.

15 **Objectives:** Small inefficiencies in freight driving strategy compound into substantial fuel losses
16 over a corridor, and improved control requires a system model. Freight dynamics are strongly
17 nonlinear and vary with both consist composition and route profile, yet no public dataset spans
18 that variation. Data-driven system identification is therefore required, which in turn requires a
19 simulation environment able to generate dynamics across a family of consists and real routes. The
20 objective of this work is to build that environment.

21 **Methods:** Classical multi-body LTD is recast into a node/edge tensor representation in which
22 vehicles are graph nodes, couplers are graph edges, and the learned component is a residual on a
23 known physics right-hand side. A reference simulator is implemented in Python in eight verified
24 stages, from a single powered vehicle through nonlinear slack couplers, N -vehicle assemblies, per-
25 vehicle route sampling, and first-order actuator lag with power-limited traction. Route profiles are
26 derived from OpenStreetMap geometry, resampled onto a uniform 10 m arc-length grid, fused with
27 USGS 3DEP elevation, and differentiated with a Savitzky-Golay filter to yield grade and curvature.

28 **Findings:** Stage-by-stage verification confirms that the simulator reproduces the phenomena dis-
29 tinguishing LTD from point-mass modeling: the deadband and sharp re-engagement of a slack
30 coupler, monotonic head-to-tail decay of steady draft force under head-end power, coupler cycling
31 driven by vehicles occupying different grades simultaneously, and divergence between commanded
32 and achieved traction under a power ceiling. On a representative 49 km corridor the pipeline recov-
33 ers a sustained climb near one percent from elevation data carrying meter-scale error, and consists
34 of 126 to 133 vehicles run end to end under randomized driving commands.

35 **Novelty:** The work contributes an openly documented, reproducible LTD data-generation testbed
36 built on real corridor terrain, together with a graph-tensor formulation that makes consist length
37 a property of the data rather than a fixed architectural dimension, which is the precondition for a
38 train-agnostic dynamical surrogate.

39 **Practical Applications:** The testbed supports energy-aware evaluation of driving strategy and
40 coupler-force safety screening across consist makeups and corridors without proprietary tools, and
41 holding out whole corridors provides direct evidence of whether a learned model transfers.

1 INTRODUCTION

2 Longitudinal train dynamics (LTD) concerns the motion and internal force distribution along the
3 length of a train. A freight consist is a distributed mechanical system of tens to hundreds of
4 coupled masses whose interactions with one another and with the track profile are strongly non-
5 linear (Cole 2006). Simulating those interactions faithfully requires solving differential equations
6 for every vehicle in the chain, and the computational burden grows steeply with model fidelity,
7 from reduced point-mass representations to full multibody dynamics (Wu et al. 2016a). At the
8 high-fidelity end the cost is severe enough that whole-trip design optimization was, before paral-
9 lelization, effectively intractable, requiring months of sequential computation (Wu et al. 2016b).

10 This computational cost is the central obstacle, since control design and operational plan-
11 ning require the model to be evaluated hundreds or thousands of times within an optimization
12 loop. A surrogate model addresses this by learning an approximate input-to-response mapping
13 that evaluates in milliseconds while retaining the underlying physical structure. Surrogates com-
14 plement rather than replace deterministic physics models: the simulator supplies the training data
15 and physical grounding, and the surrogate supplies the speed.

16 The broader project to which this paper belongs targets a specific surrogate, namely a dif-
17 ferentiable model of LTD that serves as the world model within a fuel-optimal control routine.
18 Two properties govern the design. First, the model must accommodate consists of arbitrary length,
19 because a surrogate restricted to the train lengths on which it was trained is of limited operational
20 value. Second, it must be differentiable with respect to the control plan, because the objective is
21 not to forecast a single run but to determine how the outcome shifts when traction and braking
22 commands are adjusted.

23 Before any of this can be attempted, a more elementary problem must be addressed. Train-
24 ing a surrogate that generalizes across consists and corridors requires a dataset spanning that vari-
25 ation, and no such public dataset exists. Established LTD simulators are largely proprietary, in-
26 house, or consortium-restricted (Table 1), and none is packaged to emit thousands of labeled tra-
27 jectories across randomized consists and real corridors. The prerequisite is therefore an environ-
28 ment capable of generating that data, with route profiles drawn from real track, since a surrogate
29 trained on invented grade profiles would be a weak foundation for any subsequent claim about
30 generalization to real corridors.

31 This paper documents that environment, reporting the completed preliminary phase of the
32 project and deliberately stopping short of the learned model: no surrogate is trained, and no accu-
33 racy or speedup results are claimed.

34 OBJECTIVES

35 This paper pursues four objectives, each corresponding to a completed component of the testbed.

- 36 1. To formulate the LTD prediction and control problem as a graph-tensor system in which
37 vehicles are nodes, couplers are edges, and consist length is a property of the data rather
38 than a fixed architectural dimension, and to state the learning target as a physics residual
39 that keeps the model differentiable end to end (PROBLEM FORMULATION).
- 40 2. To construct a reference LTD simulator in verifiable stages, from a single powered ve-
41 hicle to an N -vehicle consist with nonlinear slack couplers, per-vehicle route sampling,
42 actuator lag, and power-limited traction, with the boundary of its fidelity stated explic-
43 itly (REFERENCE SIMULATOR).
- 44 3. To develop an end-to-end pipeline that converts raw OpenStreetMap track geometry and

USGS 3DEP elevation into the grade, curvature, and speed-limit arrays the simulator consumes, including corridor recovery as a graph problem, uniform arc-length resampling, and Savitzky-Golay smoothing and differentiation (ROUTE ACQUISITION).

4. To provide TrainSet, a desktop research console that renders car types, trains, control profiles, and routes as named, serializable artifacts shared between interactive exploration and headless batch generation (TRAINSET).

The remainder of the paper reviews related work, states the problem formulation, and then documents each built component in turn before summarizing status and future work.

RELATED WORK

Classical Longitudinal Train Dynamics

Classical LTD studies the motion of rolling stock along the longitudinal axis of the track, accounting for both whole-train motion and the individual motion of each vehicle (Cole 2006). LTD models form a ladder of fidelity, ranging from a single point mass, through a multi-mass system in which each vehicle is a rigid body with one longitudinal degree of freedom connected by nonlinear couplers and draft gears, to models with additional degrees of freedom that expose curving effects, in-train instabilities, and derailment. Each rung trades speed for fidelity (Wu et al. 2016a); the work reported here sits deliberately on the middle rung.

The standard computational scheme treats LTD as the repeated evaluation of all force elements, namely propulsion and curving resistance, grade, traction, dynamic and air brake, and coupler forces, followed by state propagation through a numerical solver (Wu and Cole 2015). This modularity is what makes surrogate substitution feasible, since a single module can be replaced by a learned component without disturbing the rest.

The principal difficulty lies in the vehicle connection system. Friction draft gears exhibit extreme nonlinearity, including hysteresis and stick-slip, that is difficult to both model and numerically solve (Wu et al. 2016a). These force-element computations are independent across couplers and therefore parallelizable, which Wu and Cole (2015) exploit through sequential, parallel, and hybrid computing schemes. The concentration of cost in a structurally local computation is precisely what motivates a graph-based residual surrogate, because the expensive part of the model is an edge quantity.

Benchmarking has historically been weak in this field. Spiriyagin et al. (2017) introduced an international initiative defining shared input scenarios, operating conditions, and required outputs, motivated by the observation that most LTD tools evolved independently as in-house software with limited transparency into their numerical methods and assumptions. Table 1 summarizes representative tools. The pattern is consistent: capable simulators exist, but few are openly available, and none is packaged as a bulk data generator.

Data-Driven and Hybrid Modeling in Rail

Data-driven rail modeling falls into three overlapping categories. The first embeds learned components within classical multibody or finite-element frameworks, substituting nonlinear mechanical behavior with surrogates at significant runtime reduction (Nie et al. 2018) or packaging a machine-learning replacement for a full multibody model through a standardized co-simulation interface, with substantial reductions in computation time (Zhou et al. 2023). The second targets in-train forces, which are difficult to measure directly in service: supervised approaches map telemetry to coupler force estimates using causally structured convolutional networks (Zhang et al. 2024),

TABLE 1 Representative Longitudinal Train Dynamics Simulation Tools

Tool	Origin	Scope	Availability
TrainDy (Cantone et al. 2008)	UIC-oriented consortium	Freight LTD for operations and safety analysis	Specialized / consortium
TOES	Developed for AAR-member railroads (Wheel-Rail Interaction Seminar 2018)	Comprehensive train dynamics for operating scenarios	Commercial / licensed
TEDS (Federal Railroad Administration 2019)	U.S. FRA-sponsored	Safety and risk evaluation, incident investigation, operations LTD	Publicly documented
UM Train (Universal Mechanism 2024)	Universal Mechanism suite	LTD within a multibody ecosystem	Commercial
TDEAS (Wu et al. 2014)	Research and industry, China	LTD with energy analysis	Research / institutional
CRE-LTS (Spiryagin et al. 2017)	Centre for Railway Engineering, CQU	Benchmarking-ready research LTD simulator	Research / in-house

with extensions adding transfer learning to generalize across couplers and configurations (Zhang and Yan 2025). The third operates above the dynamics entirely, approximating the evaluation of candidate schedules within an optimization loop (Liu et al. 2024).

Three limitations recur and shape the present study. First, generalization across configurations remains difficult: force-estimation models are often trained on narrow scenario subsets, and even with transfer learning, performance degrades where strong nonlinear vehicle-to-vehicle interaction dominates (Zhang et al. 2024; Zhang and Yan 2025). Second, reliance on simulated labels means simulator-to-reality discrepancy compounds through a purely data-driven mapping. Third, hybrid approaches that retain physical structure generally aim to replicate nominal simulator output rather than robustness under distribution shift. In nearly every case, the training data is generated by a tool the authors do not describe in reproducible detail, which is the specific gap this paper addresses.

Learned Simulators, Graph Networks, and Sequence Surrogates

The formulation adopted here draws on work that treats physical simulation itself as a learning target with explicit relational structure. Sanchez-Gonzalez et al. (2020) demonstrate that message-passing graph networks learn particle-based physics and generalize to system sizes unseen during training. The relevance to LTD is direct, since a consist is already a graph, and one built on that structure inherits the property responsible for that generalization, namely that the computation is local and shared rather than indexed to a fixed number of entities. Neural ordinary differential equations (Chen et al. 2018) supply the complementary mechanism, allowing a learned residual

on a physics right-hand side to be integrated while remaining differentiable with respect to its inputs, the key property required by the downstream control application.

Sequence and operator models offer an alternative. Transformers replace recurrence with multi-head self-attention (Vaswani et al. 2017), and neural operators learn a solution map rather than a single-step update; both have been applied to long-horizon forecasting and to PDE-governed dynamics. These are capable tools, but they are a poorer structural fit for the present problem. A sequence model must commit to a token layout, so a consist of varying length requires padding and masking, and generalization to an unseen length is an extrapolation the architecture does not naturally support. A graph defined over vehicles and couplers obtains this property intrinsically. The project accordingly treats the graph network as the primary architecture and a transformer variant as an ablation against which the structural claim is tested.

Uncertainty as a Safety Margin

Not every simulator input is known exactly: vehicle masses, resistance coefficients, coupler stiffness, route shape, and command execution all carry uncertainty, and a substantial literature addresses this through quantile regression (Koenker and Bassett 1978) and distribution-free coverage via conformal methods (Romano et al. 2019). The present project uses that machinery narrowly. Because the surrogate is intended for fuel-optimal control rather than general-purpose forecasting, uncertainty enters primarily as a margin the controller must respect, namely keeping estimated coupler force below its limit with high probability, so the pinball loss is applied only to safety-relevant quantities, chiefly coupler force. This scope explains why the infrastructure documented below emits deterministic trajectories: parameter uncertainty is introduced by resampling the generator, not by making the simulator itself stochastic.

PROBLEM FORMULATION

The physics presented below is standard; what matters is how it is organized, because the simulator’s outputs, the surrogate’s training targets, and the surrogate’s internal state all share a single layout regardless of the number of vehicles in the train.

Classical Formulation

Represent the train as a chain of N vehicles indexed $i = 0, \dots, N - 1$ from lead to tail, connected by $N - 1$ couplers. Let $x_i(t)$ and $v_i(t) = \dot{x}_i$ denote the longitudinal position and velocity of vehicle i along the route, and let m_i denote its mass. Between vehicles i and $i + 1$, coupler i has extension δ_i (relative displacement from nominal spacing, positive in draft) and transmits force F_i to the trailing vehicle. Collecting the external and control contributions on vehicle i into $F_{\text{ext},i}(t)$,

$$\dot{x}_i = v_i, \tag{1}$$

$$m_i \dot{v}_i = F_{\text{ext},i}(t) + F_{i-1}(t) - F_i(t), \tag{2}$$

with $F_{-1} = F_{N-1} = 0$ when end forces are absent. Coupler forces follow constitutive laws $F_i = \phi_i(\delta_i, \dot{\delta}_i, \dots)$, in which slack, asymmetric buff and draft stiffness, and damping enter; actuator lag and power limits introduce further per-vehicle states.

Tensorized State Representation

For learning, and for software that must accommodate variable train length, the same physics is organized into two arrays that make the vehicle-coupler graph explicit. A node tensor $\mathbf{H}(t) \in$

1 $\mathbb{R}^{N \times d_n}$ collects d_n channels per vehicle: position, velocity, internal actuator states where those
 2 are dynamic, and static attributes replicated per row (mass, Davis coefficients, traction capability,
 3 and force limits). An edge tensor $\mathbf{E}(t) \in \mathbb{R}^{(N-1) \times d_e}$ holds d_e channels per coupler: extension δ_i ,
 4 relative speed $\dot{\delta}_i$, coupler force F_i , nominal spacing, slack half-width, and buff and draft stiffness
 5 and damping parameters. Over a discrete time grid, trajectories are stored as rollout tensors $\mathbf{X} \in$
 6 $\mathbb{R}^{T \times N \times d_n}$ and $\mathbf{E}_{\text{hist}} \in \mathbb{R}^{T \times (N-1) \times d_e}$.

7 This layout keeps the structure of the physical problem visible, since each coupler row
 8 depends only on the two adjacent vehicle rows, mirroring the chain structure of Equation 2. For
 9 this reason the train is treated as a graph rather than as a fixed-length sequence. A graph-based
 10 model processes trains of any length directly, passing information between neighboring vehicles
 11 one coupler at a time, with no padding or masking. Handling variable length as a consequence
 12 of the representation rather than as an engineering workaround is what makes the train-agnostic
 13 objective attainable.

14 **Dynamics and the Learning Target**

Let $\mathbf{H}_{\text{dyn}}(t)$ denote the dynamic channels of $\mathbf{H}(t)$ and let $\mathbf{p}$ denote the fixed parameters. The differential part of the evolution is

$$\frac{d}{dt} \text{vec}(\mathbf{H}_{\text{dyn}}(t)) = \mathcal{F}_n(\mathbf{H}(t), \mathbf{E}(t), \mathcal{U}(t), \mathcal{R}; t), \quad (3)$$

where $\text{vec}(\cdot)$ stacks the columns of its matrix argument into a single column vector, $\mathcal{U}$ denotes the control inputs, $\mathcal{R}$ the route profile, and $\mathcal{F}_n$ the assembled right-hand side obtained from Equation 2 together with the actuator relations. In the reference integrator, several edge channels, including δ_i , $\dot{\delta}_i$, and F_i , are reconstructed algebraically at each evaluation from vehicle positions and velocities rather than integrated as separate states, so that

$$\mathbf{E}(t) = \mathcal{G}_e(\mathbf{H}(t), \mathbf{p}) \quad (4)$$

15 for those channels, while Equation 3 carries the true differential degrees of freedom. Here $\mathcal{G}_e$
 16 denotes the algebraic reconstruction map defined by the coupler kinematics and constitutive laws,
 17 for example $\delta_i = (x_i - x_{i+1}) - L_{0,i}$ together with the associated force law. Were internal draft-gear
 18 states introduced in later work, Equation 4 would be supplemented by differential equations on the
 19 corresponding rows.

20 The surrogate does not learn $\mathcal{F}_n$ from a blank slate. It begins from the known physics right-
 21 hand side and learns a correction, so that the predicted rate of change takes the form $\mathcal{F}_n + \Delta \mathcal{F}_\theta$,
 22 where $\Delta \mathcal{F}_\theta$ is a message-passing graph network operating on $(\mathbf{H}, \mathbf{E})$ and the whole is integrated
 23 as a neural ODE (Chen et al. 2018). Building physics into the model structurally, rather than
 24 supplying it as an input feature or as a soft penalty in the loss, anchors the surrogate’s behavior for
 25 consists and routes it has not encountered.

26 **Inputs, Outputs, and Objective**

27 Every scenario is specified by a triple $(\mathcal{R}, \mathcal{C}, \mathcal{U})$: a route profile $\mathcal{R}$, defined as a piecewise field
 28 over distance s supplying grade $\sin \theta(s)$, curvature proxy $\kappa(s)$, and speed limit $v_{\text{max}}(s)$, evaluated
 29 per vehicle at $s = x_i(t)$ and produced by the pipeline of ROUTE ACQUISITION; a consist $\mathcal{C}$,
 30 comprising two aligned tables of per-vehicle and per-coupler attributes that map directly onto the
 31 static channels of $\mathbf{H}$ and $\mathbf{E}$; and a control plan $\mathcal{U}$, supplying commands $u_{\text{trac},i}(t)$ and $u_{\text{brk},i}(t)$ that
 32 drive the actuator dynamics.

The reference physics implements a deterministic operator

$$F : (\mathcal{R}, \mathcal{C}, \mathcal{U}) \mapsto \mathbf{y}, \quad \mathbf{y} = (\mathbf{X}, \mathbf{E}_{\text{hist}}, \mathbf{s}, \mathbf{r}), \quad (5)$$

where $\mathbf{s}$ holds scalar summaries and $\mathbf{r}$ holds risk-oriented statistics. The summaries used throughout are trip energy, arrival time, and peak coupler force,

$$E_{\text{trip}} = \int_0^T P_{\text{trac}}(t) dt, \quad (6)$$

$$T_{\text{arr}} = \inf\{t : x_{\text{lead}}(t) \geq s_{\text{target}}\}, \quad (7)$$

$$F_{\text{max}} = \max_{t,j} |F_j(t)|, \quad (8)$$

each a post-processing map on the rollout tensors, where s_{target} is the target arc-length position of the destination along the route. Training data form the collection

$$\mathcal{D} = \{(\mathcal{R}^{(k)}, \mathcal{C}^{(k)}, \mathcal{U}^{(k)}, \mathbf{X}^{(k)}, \mathbf{E}_{\text{hist}}^{(k)}, \mathbf{s}^{(k)})\}_{k=1}^M, \quad (9)$$

and the surrogate f_θ is fit against a weighted objective

$$\mathcal{L}(\theta) = \lambda_X \mathcal{L}_X + \lambda_E \mathcal{L}_E + \lambda_S \mathcal{L}_S + \lambda_Q \mathcal{L}_Q + \lambda_P \mathcal{L}_P, \quad (10)$$

- 1 combining rollout losses on the node and edge tensors, a summary loss on the trip-level scalars,
- 2 a pinball loss $\mathcal{L}_Q$ applied narrowly to the safety-relevant coupler-force margin, and a small, op-
- 3 tional physics-consistency penalty $\mathcal{L}_P$ that detects residual inconsistencies such as a coupler-force
- 4 channel disagreeing with the accelerations of the two vehicles it connects. Because the residual
- 5 architecture already builds in most of the physical plausibility, $\mathcal{L}_P$ carries little weight.

- 6 Evaluation will be defined on these same outputs: trajectory fidelity channel by channel,
- 7 summary accuracy, safety-margin quality, generalization to held-out consist lengths and corridors,
- 8 and inference speedup. Everything from this section forward is what must exist before any of those
- 9 quantities can be produced.

10 REFERENCE SIMULATOR

- 11 Every training example is produced by running the simulator once, so this component is the data
- 12 generator for the entire project. It was constructed one physical effect at a time, so that each
- 13 addition could be verified before the next was introduced. The implementation follows a notebook
- 14 that builds the model stage by stage using NumPy, SciPy's adaptive `solve_ivp` integrator, and
- 15 Matplotlib, and is mirrored in a structured Python package that exposes the node and edge tensors
- 16 alongside the flat state used for integration.

- 17 One caveat governs the interpretation of everything that follows. The parameters used,
- 18 including vehicle mass, resistance coefficients, coupler stiffness, and traction limits, are realistic in
- 19 form but are not tuned to any specific real locomotive or railcar. The goal at this stage is a simulator
- 20 whose structure is correct, not one whose numbers are certified against a test train; accordingly,
- 21 the surrogate and control experiments built upon it are validated against the simulator rather than
- 22 against real-world rail data.

23 Stages 1 and 2: Single Vehicle and Linear Coupler

The starting point is a single point mass with the force balance

$$m\dot{v} = F_{\text{trac}}(t) - F_{\text{brk}}(t) - R(v) - G(x) - C(x, v), \quad (11)$$

with $\dot{x} = v$ and Davis running resistance in magnitude-direction form,

$$R(v) = (A + B|v| + Cv^2) \operatorname{sgn}(v), \quad (12)$$

where $\operatorname{sgn}(v)$ is set to zero for $|v| < \varepsilon$ to avoid chatter at standstill, grade enters as $G(x) = mg \sin \theta(x)$, and an optional curvature proxy C is proportional to $mv^2|\kappa(x)|$. Traction commands use a smooth ramp rather than an instantaneous step, which eases numerical stiffness and anticipates the actuator-lag extension. A single-vehicle run on level track, with braking and curvature disabled, confirms the integrator and force-sign behavior before couplers are introduced.

Attaching a second mass at nominal center-to-center spacing L_0 introduces the coupler extension $\delta = (x_0 - x_1) - L_0$ and rate $\dot{\delta} = v_0 - v_1$, with a linear spring-damper exerting $F = k\delta + c\dot{\delta}$ on the rear car and $-F$ on the lead. For stiff (k, c) the two speed traces nearly coincide, even though the coupler force and extension carry the load path, which is an early indication that speed alone is a poor diagnostic for this system.

Stage 3: Nonlinear Coupler with Slack

The linear law is replaced by a piecewise law with slack half-width s . For $|\delta| \leq s$ the coupler force is zero. For $\delta > s$, draft stiffness and damping (k_d, c_d) act on the over-travel $\delta - s$; for $\delta < -s$, a distinct buff pair (k_b, c_b) acts on $\delta + s$. Figure 1 overlays the linear force that the Stage 2 parameters would produce against the actual nonlinear force along the same trajectory. The deadband produces intervals of near-zero force followed by sharper loading when contact re-engages, which is the mechanism behind impact-style longitudinal loads and the reason slack-aware modeling cannot be omitted.

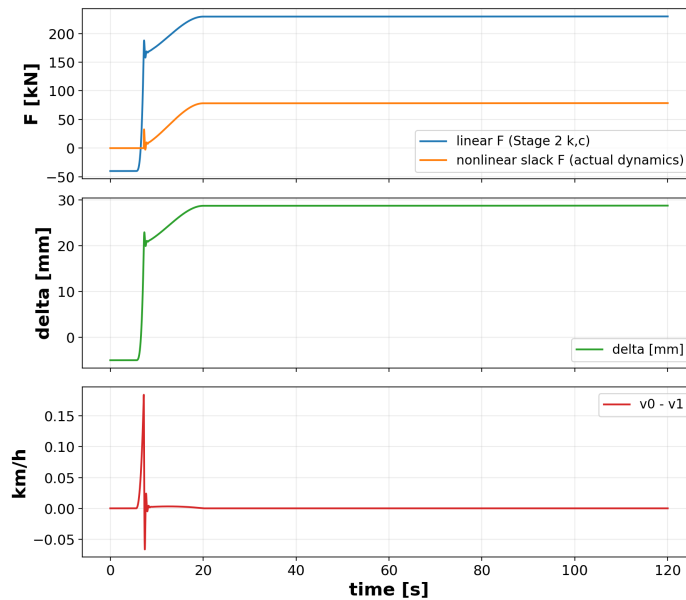


FIGURE 1 Stage 3 comparison of the linear spring-damper force against the nonlinear slack coupler force along the same trajectory, with extension and relative speed shown below. The linear force is nonzero at rest, whereas the slack force remains at zero through the deadband.

1 Stages 4 and 5: Chains of Arbitrary Length

With vehicles indexed front to rear and coupler j connecting j to $j + 1$, the recursion generalizes directly. Stacking the state as $\mathbf{y} = (x_0, \dots, x_{N-1}, v_0, \dots, v_{N-1})^\top \in \mathbb{R}^{2N}$, the right-hand side for vehicle i is

$$m_i \dot{v}_i = F_{\text{trac},i} - F_{\text{brk},i} - R_i(v_i) - G_i(x_i) - C_i + F_{i-1}^{\text{in}} - F_i^{\text{out}}, \quad (13)$$

- 2 with $F_{-1}^{\text{in}} = F_{N-1}^{\text{out}} = 0$, and with traction and braking set to zero on unpowered cars. At this stage
 3 the simulator ceases to be tied to a fixed train length, since a single code path handles any N , which
 4 later permits training data spanning a wide range of consist sizes. Figure 2 shows all coupler forces
 5 as a heatmap over time and coupler index, exposing the monotonic decay of steady draft force from
 6 head to tail when only the lead provides traction.

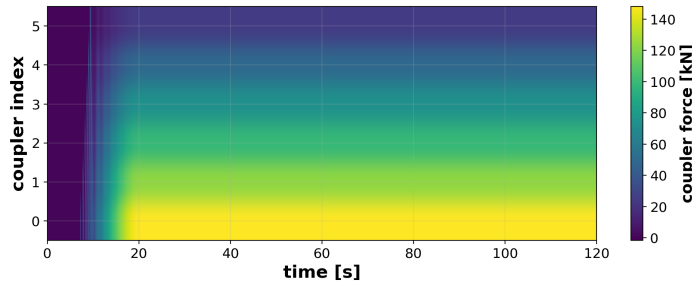


FIGURE 2 Stage 5 coupler force across an N -vehicle consist, plotted against time and coupler index, showing steady draft force decaying monotonically from head to tail under head-end power.

7 Stage 6: Per-Vehicle Grade and Curvature

- 8 Grade and curvature are no longer evaluated at a single global position. Each vehicle samples
 9 $\sin \theta(x_i)$ and, optionally, $\kappa(x_i)$ at its own location, so that $G_i = m_i g \sin \theta(x_i)$. This introduces
 10 spatial dispersion along the train, since head and tail may occupy different grades simultaneously,
 11 exciting relative motion and coupler cycling even where speed traces remain nearly overlaid. For
 12 a consist more than a kilometer long this is not a second-order effect, and it is where route data
 13 quality begins to bear directly on the dynamics.

14 Stage 7: Actuator Dynamics and Power-Limited Traction

The state is augmented by first-order lag variables for the commanded brake and traction levels on each vehicle:

$$\dot{z}_{\text{brk},i} = \frac{u_{\text{brk},i} - z_{\text{brk},i}}{\tau_b}, \quad \dot{z}_{\text{trac},i} = \frac{u_{\text{trac},i} - z_{\text{trac},i}}{\tau_t}. \quad (14)$$

Achieved brake force is capped at a per-vehicle limit. For powered vehicles, achieved traction follows a continuous power cap,

$$F_{\text{trac},i} = \min \left(\max(z_{\text{trac},i}, 0), \frac{P_{\text{max}}}{\max(|v_i|, \varepsilon)} \right), \quad (15)$$

- 15 with P_{max} an illustrative constant power ceiling and ε a small speed regularizer. Figure 3 shows
 16 lead speed together with the lag states for a scenario in which traction and braking commands

1 overlap; the plotted states are commands, not achieved forces, which additionally reflect Equa-
 2 tion 15 and brake saturation. This stage makes the control plan $\mathcal{U}$ physically meaningful rather
 3 than instantaneous, a precondition for the control application the surrogate is intended to serve.

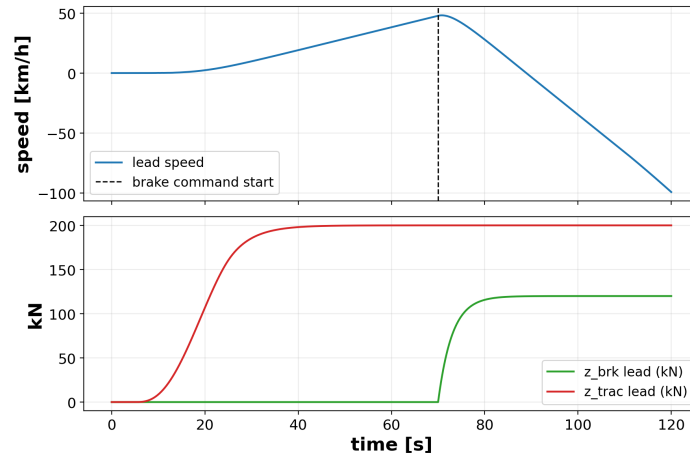


FIGURE 3 Stage 7 lead speed with the brake-command onset marked, together with the first-order lag states for the brake and traction commands on the lead vehicle.

4 **Stage 8: Scope Relative to Industrial High Fidelity**

5 Several effects commonly associated with high fidelity are deliberately outside this code path,
 6 including hysteretic draft gear, pneumatic brake pipe dynamics, distributed adhesion limits, and
 7 calibration to field data. These effects mark the boundary between the intermediate-fidelity sim-
 8 ulator used here and a fully industrial toolchain, and it is stated rather than concealed because it
 9 bounds every claim the project can make.

10 **ROUTE ACQUISITION AND TERRAIN DATA PIPELINE**

11 The simulator treats the route profile as a given input; this section documents where that input
 12 originates. The governing decision is that routes are built from real terrain rather than invented,
 13 because a surrogate is only as convincing as the data behind it. Satisfying that decision requires
 14 pulling raw track geometry from an open mapping database, converting it into clean continuous
 15 corridors, attaching real elevation, and producing exactly the fields the simulator expects. The
 16 pipeline runs in five stages, namely acquire, resample, elevation, profile, and export, each with
 17 its own storage and a corresponding tab in the route-acquisition console. It is designed to run in
 18 batches rather than to process a national network in a single pass.

19 **Acquisition Must Be Area-First**

20 Railway lines are not tidy objects in OpenStreetMap (OpenStreetMap contributors 2026). Track
 21 is mapped as many short, disconnected segments called ways, and although OSM defines route
 22 relations intended to group ways into named lines, coverage for United States freight corridors
 23 is incomplete and a given relation does not reliably correspond to one clean, walkable corridor.
 24 Rendering the mapped track around Pueblo, Colorado makes the problem plain: mainline corridors
 25 run through a dense tangle of yard and service track and meet at several junctions, and no single
 26 map object corresponds to the line through Pueblo.

1 Selection is therefore area-first. A user specifies a bounding box or a radius around a point,
2 all track ways inside the area are retrieved, and the resulting tangle is split back into corridors
3 algorithmically. Fetching by relation identifier is retained as a secondary path for cases in which a
4 well-formed relation happens to exist.

5 **Corridor Extraction as a Graph Problem**

6 Track ways are assembled into a graph whose vertices are track endpoints and junctions and whose
7 edges are way segments. Every vertex takes one of three roles: a continuation node has degree two
8 and passes a line through, a line-end has degree one and marks a termination, and a junction has
9 degree three or higher and marks a genuine branch point. Corridors are extracted as maximal paths
10 through continuation nodes, breaking at line-ends and junctions, which converts an unordered
11 collection of fragments into distinct, traceable lines.

12 For radius queries, an optional bounded outward crawl follows each line's open ends past
13 the edge of the original radius, issuing additional queries just beyond them until each line reaches
14 a natural end or a hard cap on distance, iteration count, or total ways is reached. This captures
15 lines that merely pass through the queried area, while the cap prevents expansion into the fully
16 connected national network.

17 OSM splits a single physical line into many ways wherever a junction, bridge, or tagging
18 change occurs, and sidings fragment lines further, so a long corridor can arrive as dozens of discon-
19 nected pieces. A grouping step re-links collinear fragments across both real junctions and small
20 unshared endpoint gaps, using endpoint proximity together with heading continuity. Nothing is
21 merged or discarded: fragments retain their identity and remain reversible, and are simply labeled
22 with a shared line identifier and an order along that line. Ways tagged as sidings, spurs, yards,
23 or crossovers are flagged as service track and excluded. A related routine builds a robust center-
24 line from the member ways, stitching unordered and reversed segments into one ordered polyline,
25 detecting spatial gaps and branch points explicitly rather than silently choosing a continuation,
26 carrying per-vertex tags, and computing geodesic arc length.

27 A Streamlit application exposes both acquisition modes. In area mode, a user retrieves a
28 region and views the extracted corridors color-coded on a map alongside a sortable table and per-
29 corridor quality checks, which include gap count and size, branch points, speed-limit coverage,
30 and the fraction of the line on bridges or in tunnels; selected corridors are then added to an export
31 batch.

32 **Resampling to a Uniform Grid**

33 OSM digitizes track adaptively with respect to horizontal curvature, placing dense vertices on
34 curves and sparse vertices on tangents. That pattern under-samples exactly the straight stretches
35 where the vertical profile still requires accurate capture, so each stored route is rewritten onto a
36 fixed arc-length grid, 10 m by default, by linearly interpolating longitude and latitude along arc
37 length.

38 Two details merit attention. First, step-valued features, namely speed limit, way identifier,
39 and the bridge, tunnel, cutting, and embankment flags, are not smoothly interpolated; each resam-
40 pled point inherits the value of whichever source segment contains it, and a sample is inserted
41 exactly where such a feature changes, so that a speed-limit step or a bridge entrance is not blurred
42 across a grid cell. Second, the stage validates its own output: coincident vertices are dropped, arc
43 length is enforced to be strictly increasing, and each point is tagged with its source segment length

1 and a flag marking interpolation across an unusually long or bridged gap, so that stretches where
 2 the geometry is inferred rather than mapped remain visible downstream. A representative 49.1 km
 3 corridor rewritten onto the uniform grid yields 4,920 points, 713 of them interpolated across long
 4 source segments.

5 **Elevation Fusion**

6 Each resampled point receives a ground elevation from the USGS 3DEP digital elevation model
 7 (U.S. Geological Survey 2022), queried in batches against the 3DEP image-service endpoint with
 8 a slower per-point service as a fallback and retries with backoff when the service is busy. Points
 9 whose elevation cannot be determined are left missing rather than guessed, and every earlier per-
 10 vertex field is preserved. Rendering the result in three dimensions catches corridors that have been
 11 stitched incorrectly or that jump between parallel alignments before any derivative is taken.

12 **Deriving the Route Profile**

13 This stage produces the fields the simulator consumes. On a bridge or in a tunnel, the DEM reports
 14 the elevation of the structure or the ground above it rather than of the rail, and sometimes reports
 15 nothing; these stretches are identified from the structural flags carried since acquisition, discarded,
 16 and filled by linear interpolation.

Raw DEM elevation carries roughly a meter of error, which is small against elevation it-
 self but large against its rate of change over a 10 m step. Differentiating the raw signal directly
 would therefore yield a grade estimate that is almost entirely noise. Instead, a Savitzky-Golay
 filter (Savitzky and Golay 1964) fits a local polynomial through a moving window, yielding a
 smoothed elevation and its analytic derivative from the same operation rather than smoothing and
 differentiating in two error-compounding steps. Grade follows as $\sin \theta = \sin(\arctan(dz/ds))$ with
 a physically motivated clamp. Curvature is derived in the same manner, projecting the horizontal
 coordinates into a local planar frame so that the first and second filter derivatives give

$$\kappa = \frac{x'y'' - y'x''}{(x'^2 + y'^2)^{3/2}}, \quad (16)$$

17 clamped to a minimum turning radius. Speed limit is converted from the map's tagged value to
 18 meters per second where available, falling back to an estimate based on Federal Railroad Admin-
 19 istration track class where OSM coverage is missing.

20 Figure 4 shows the three derived fields for the same corridor, whose raw elevation climbs
 21 from roughly 1,430 m to 1,740 m over 49 km. The noisier grade over the first 20 km corresponds
 22 to the urban approach, where frequent bridges, junctions, and short mapped segments degrade
 23 the elevation signal; beyond it the line settles onto a sustained climb near one percent. That the
 24 degradation is confined to a stretch with an identifiable physical cause, rather than distributed along
 25 the line, is itself a useful diagnostic.

26 **Export and Reliability**

27 Finished routes are collected into an export bundle: one array file per route holding exactly
 28 `route_s`, `route_sin_theta`, `route_kappa`, and `route_vmax`; an index of each route's location,
 29 endpoints, length, and grade and curvature statistics; and a manifest recording the meaning and
 30 provenance of each channel. The array files map directly onto the simulator's route-profile input,
 31 and each is sampled through an actual route-profile object as a final check before it is trusted.



FIGURE 4 The route-profile stage of the acquisition console, which derives the fields the simulator consumes from the elevated samples. The controls at top set the elevation smoothing window, grade clamp, minimum curve radius, and Savitzky-Golay polynomial order; the panels below show, for the same 49 km corridor, the raw and smoothed elevation, the grade $\sin \theta(s)$ as a percentage, the curvature proxy $\kappa(s)$, and the step-valued speed limit $v_{\max}(s)$. The noisier grade over the first 20 km corresponds to the urban approach.

Because pulling all track in a large or dense area in a single request tends to time out, queries exclude service and industrial spurs, the seed query is split into a grid of small requests, and every request retries with exponential backoff and jitter across mirror servers. For a handful of routes the interactive application suffices; for a larger dataset, the pipeline is first verified interactively on known areas, confirming that lines render as continuous corridors, that gaps are short and located at bridges, and that traced length is plausible against an independent map. Only then is a headless batch crawler run over curated seed points, deduplicating overlapping pulls and resuming from a cache after interruption.

Two design choices run through the pipeline. First, acquisition and assembly are pure functions wherever they do not require a live network call, which makes the line-stitching logic unit-testable offline against a fixture built from a captured query response. Second, data quality problems are surfaced rather than silently patched: gaps are bridged but their size reported, and branch points flagged rather than auto-resolved, so the user decides whether a route is clean enough to use.

15 TRAINSET: A DESKTOP RESEARCH CONSOLE

The simulator and the route pipeline are libraries, driven from notebooks and scripts, with consists, routes, and command sets assembled by hand as Python objects. This is a workable way to produce a figure but an inefficient basis for exploration, which constitutes much of the surrounding research.

1 TrainSet is a desktop application built over the same backend. It is not a deliverable of the study in
2 its own right, and no numerical claim depends on it; it is documented here because the way a tool
3 shapes which questions are asked is part of the method.

4 **Motivation and Architecture**

5 Three considerations motivated its construction. The first is the cost of a question: determining
6 what happens to coupler forces if a train is thirty cars longer on a particular grade requires, in script
7 form, editing a consist definition, rebuilding a scenario, re-running the solver, and re-plotting, so
8 that many such questions are never asked. The second is the gap between a number and a shape,
9 since the failure modes that matter here, such as a run-in resolved unphysically, a grade artifact
10 surviving the smoothing stage, or a brake command overlapping traction in a manner no operator
11 would produce, are immediately visible in a plot yet nearly invisible in a summary statistic. The
12 third is reproducible bookkeeping, since every object involved must be nameable, saveable, and
13 reloadable so that a run can be repeated exactly.

14 The governing architectural constraint is that the user interface never imports the numer-
15 ical stack. The backend packages are reached only through a thin service layer, so the backend
16 continues to run headless: notebooks, the batch generator, and the figures in this paper have no
17 dependency on the interface toolkit, and each service is unit-testable without a display. The bound-
18 ary is also a natural insertion point, since the surrogate and controller attach as additional services
19 without altering any existing screen. Because the simulator integrates the whole time span in one
20 adaptive solver call, there is no intermediate hook against which to report progress or to interrupt,
21 and the stop control is correspondingly disabled, a genuine limitation rather than an oversight,
22 since streaming a run requires a stepwise integrator that the reference implementation deliberately
23 does not use.

24 **Screens and Artifacts**

25 The route screen browses the output of ROUTE ACQUISITION, rendering chainage-linked eleva-
26 tion, grade, and curvature plots alongside a summary panel of length, elevation range, climb and
27 descent, grade range, minimum radius, speed-limit range, and the fraction of the line for which the
28 map actually supplied a speed limit. That last field is a data-quality property rather than a physical
29 one, easy to overlook when a route is loaded programmatically but impossible to overlook when
30 printed beside the grade range on every load.

31 The consist screen implements the three-way split around which the data model was de-
32 signed: a library of reusable car types, an ordered train under construction, and a shelf of saved
33 trains (Figure 5a). A car type is a named template carrying mass split into tare and lading, Davis
34 coefficients, traction and braking ceilings, physical length, and a coupler type, stored one per file
35 as a plain-text record so that the library is version-controllable outside the application. A train is
36 an ordered list of references to those types, so that a train file stays small regardless of length and a
37 correction to a car type propagates to every train that uses it. Any car may override selected fields
38 while inheriting the rest, which makes a train with a single unusually heavy car expressible without
39 introducing a near-duplicate type.

40 The consist in Figure 5a was produced by the procedural generator rather than assembled
41 by hand. The generator writes through the same save path the interface uses, so generated and
42 hand-built trains are indistinguishable downstream, which makes large training sweeps possible
43 without a second code path. The controls screen generates driving commands. A control profile

1 is a train-agnostic recipe, defined as piecewise command curves expressed as a fraction of each
2 vehicle's own force ceiling rather than in newtons, so that one profile drives any consist, with knots
3 interpolated using the same smoothstep shape as the simulator's traction ramp to avoid the stiffness
4 that instantaneous steps introduce. Profiles are generated randomly under user-specified bounds on
5 duration, knot count, and traction and brake fraction ranges, with a live preview of the resulting
6 command for each power role. Distributed power units are addressed by role, namely head-end,
7 mid-train, and rear helper, resolved from each car's label, and may be driven synchronously with a
8 configurable lag behind the head end, independently, or not at all. Generation is seeded, so that a
9 profile is reproducible from its configuration alone.

10 Randomized commands are what the surrogate's training set requires, since a model trained
11 only on hand-designed driving would never encounter the transients that make coupler dynamics
12 difficult, while the bounded ranges keep the randomness plausible rather than arbitrary.

13 The simulate screen is deliberately spare: it selects a saved train and route, sets a duration
14 and two numerical options, selects a control profile, and runs. A run is therefore fully specified
15 by a short list of named artifacts and four numbers, and is trivially repeatable. Results plot every
16 vehicle's speed above every coupler's force on time-linked axes (Figure 5b). Plotting every vehicle
17 rather than a summary is intentional, since the information is carried by the spread between traces,
18 with the speed envelope widening and collapsing as slack runs in and out. That behavior is invisible
19 in a mean, and the qualitative signature, namely oscillation that decays rather than grows and forces
20 that remain bounded, is a check that no summary statistic provides.

21 **Status and Limitations**

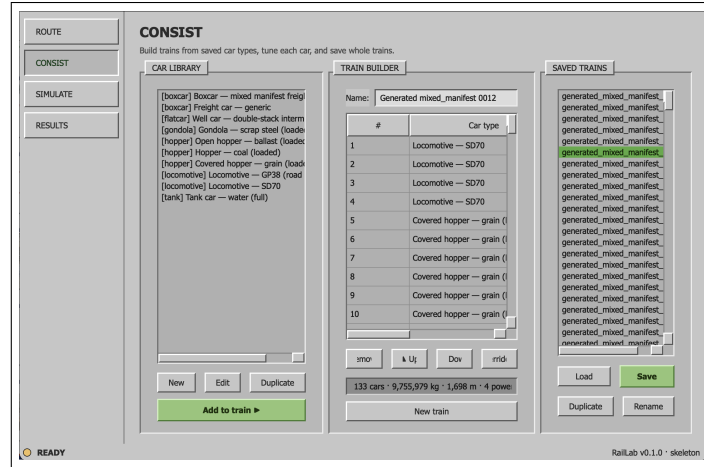
22 TrainSet is a working instrument rather than finished software. Runs cannot be interrupted or mon-
23 itored mid-flight because the solver call is atomic, and the results screen does not yet expose the
24 full node and edge state or export a run directly; dataset generation remains the responsibility of
25 the headless batch path, which is the correct division of labor, since the interface is intended for
26 examining a single run and the batch runner for producing thousands. Most importantly, nothing in
27 the application alters the standing caveat of REFERENCE SIMULATOR: the car-library parame-
28 ters are realistic in form but are not calibrated to any specific rolling stock, and the ease of building
29 a train and observing its behavior should not be mistaken for validation.

30 **CONCLUSIONS AND FUTURE WORK**

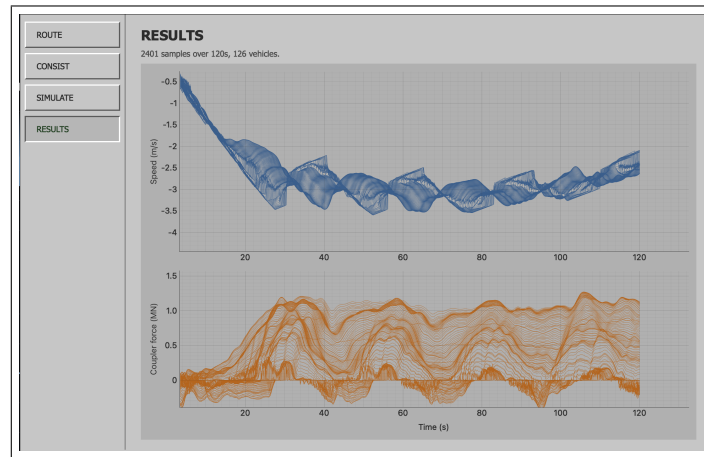
31 The three components compose into a single loop: a corridor is acquired from real map and eleva-
32 tion data and reduced to four arrays, a consist is assembled from a reusable car library, a random-
33 ized but bounded control profile is generated as a train-agnostic recipe, and the simulator integrates
34 the result into rollout tensors in the layout of PROBLEM FORMULATION. Because the console
35 and the batch generator share one artifact format, a run inspected interactively and a run within a
36 million-sample training set are the same object.

37 This design enables the generalization experiment that the project's central claim requires.
38 Consist length is a property of the data rather than an architectural dimension, so training on one
39 range of lengths and evaluating on another requires no change to the model; and because routes
40 are drawn from real corridors, whole corridors can be held out, a substantively different test from
41 holding out synthetic variations of a single profile.

42 Three limitations bound what the testbed supports. First, the simulator sits at intermediate
43 fidelity, with hysteretic draft gear, brake pipe dynamics, and adhesion limits outside its scope. Sec-



(a)



(b)

FIGURE 5 The TrainSet console. (a) The consist screen: a library of reusable car types (left), the ordered train under construction with per-car overrides (center), and saved trains (right); the train shown is a procedurally generated mixed manifest of 133 cars, roughly 9,756 tonnes and 1,698 meters long, with four powered units at the head. (b) The results screen after a 120 s run of a 126-vehicle consist under a randomized control profile: every vehicle's speed is plotted above every coupler's force on time-linked axes, and the widening and collapsing speed envelope reflects slack running in and out, with the corresponding fan of coupler forces below.

1 ond, its parameters are illustrative rather than calibrated, so that every downstream claim concerns
 2 fidelity to this simulator rather than to real rolling stock. Third, route data quality is inherited
 3 from open sources, where DEM-derived grade is trustworthy in aggregate but degrades on urban
 4 approaches, which is why the pipeline surfaces coverage and interpolation statistics rather than
 5 smoothing them away.

6 Work proceeds along three lines. The immediate step is bulk dataset generation across a
 7 curated set of corridors and a randomized family of consists, with held-out corridors and consist-
 8 length ranges reserved before training begins. Next is the surrogate itself: a message-passing graph

network producing the residual $\Delta \mathcal{F}_\theta$ of Equation 3, integrated as a neural ODE and trained against Equation 10, and benchmarked against physics-only and physics-residual multilayer-perceptron baselines as well as a transformer variant that tests whether the graph structure earns its place. Finally, the trained surrogate becomes the differentiable world model within a model-predictive control routine minimizing trip energy on held-out routes subject to speed limits and coupler-force constraints.

No surrogate is trained in this paper, and no accuracy or speedup is claimed. What is established is that the phenomena distinguishing LTD from point-mass modeling are reproduced on corridors derived from real track, at consist sizes above 120 vehicles, under randomized driving commands, and in a form that can be generated in bulk and held out by corridor, the precondition for the modeling and control results to follow.

ACKNOWLEDGMENTS

The authors thank the School of Engineering at Colorado State University Pueblo. Route geometry is derived from OpenStreetMap, © OpenStreetMap contributors, available under the Open Database License; elevation data are from the U.S. Geological Survey 3D Elevation Program. This work was supported by the Federal Railroad Administration’s Consolidated Rail Infrastructure and Safety Improvements (CRISI) Program through a railway workforce development project led by the Center for Urban Transportation Research at the University of South Florida.

AUTHOR CONTRIBUTIONS

The authors confirm their contributions to the paper as follows. George James: conceptualization, methodology, software, data curation, formal analysis, visualization, and writing of the original draft. Trung Duong: conceptualization, methodology, supervision, validation, and review and editing. Md Islam: funding acquisition, project administration, resources, supervision, and review and editing. Saqib Gulzar: project administration, supervision, and review and editing. All authors reviewed and approved the final manuscript.

DECLARATION OF CONFLICTING INTERESTS

The authors declared no potential conflicts of interest with respect to the research, authorship, or publication of this article.

REFERENCES

- Cole, C., Longitudinal Train Dynamics. In *Handbook of Railway Vehicle Dynamics* (S. Iwnicki, ed.), CRC Press, Boca Raton, FL, 2006, pp. 239–278.
- Wu, Q., M. Spiryagin, and C. Cole, Longitudinal Train Dynamics: An Overview. *Vehicle System Dynamics*, Vol. 54, No. 12, 2016a, pp. 1688–1714.
- Wu, Q., C. Cole, and M. Spiryagin, Parallel Computing Enables Whole-Trip Train Dynamics Optimizations. *Journal of Computational and Nonlinear Dynamics*, Vol. 11, No. 4, 2016b, p. 044503.
- Wu, Q. and C. Cole, Computing Schemes for Longitudinal Train Dynamics: Sequential, Parallel and Hybrid. *Journal of Computational and Nonlinear Dynamics*, Vol. 10, No. 6, 2015, p. 064502.
- Spiryagin, M., Q. Wu, and C. Cole, International Benchmarking of Longitudinal Train Dynamics

- 1 Simulators: Benchmarking Questions. *Vehicle System Dynamics*, Vol. 55, No. 4, 2017, pp.
2 450–463.
- 3 Cantone, L., L. Müller, D. Negretti, and V. Vullo, TrainDy, a New UIC Simulator for the Longi-
4 tudinal Dynamics of Trains. In *Computers in Railways XI*, WIT Press, Southampton, UK,
5 2008, Vol. 103 of *WIT Transactions on The Built Environment*, pp. 735–744.
- 6 Wheel-Rail Interaction Seminar, *Vehicle-Track Modeling and Simulation*. Wheel-Rail Interaction
7 Conference presentation, 2018, accessed February 2, 2026.
- 8 Federal Railroad Administration, *Train Energy and Dynamics Simulator (TEDS): Current*
9 *Projects*. <https://railroads.dot.gov>, 2019, accessed February 2, 2026.
- 10 Universal Mechanism, *UM Train: Module for Train Dynamics Simulation*. <https://www.universa>
11 [lmechanism.com](https://www.universallmechanism.com), 2024, accessed February 2, 2026.
- 12 Wu, Q., S. Luo, and C. Cole, Longitudinal Dynamics and Energy Analysis for Heavy Haul Trains.
13 *Journal of Modern Transportation*, Vol. 22, No. 3, 2014, pp. 127–136.
- 14 Nie, Y., Z. Tang, F. Liu, J. Chang, and J. J. Zhang, A Data-Driven Dynamics Simulation Framework
15 for Railway Vehicles. *Vehicle System Dynamics*, Vol. 56, No. 3, 2018, pp. 406–427.
- 16 Zhou, S., A. Meierhofer, O. Kugu, Y. Xia, and M. Grafinger, A Machine-Learning-Based Surrogate
17 Modeling Methodology for Submodel Integration in the Holistic Railway Digital Twin
18 Platform. In *Procedia CIRP (33rd CIRP Design Conference)*, Elsevier, 2023, Vol. 119, pp.
19 345–350.
- 20 Zhang, S., P. Huang, T. Constable, and W. Yan, A Data-Driven Approach for Railway In-Train
21 Forces Monitoring. *Advanced Engineering Informatics*, Vol. 59, 2024, p. 102258.
- 22 Zhang, S. and W. Yan, In-trainNet: A Two-Step Data-Driven Framework for Enhancing Railway
23 In-Train Forces Monitoring. *Advanced Engineering Informatics*, Vol. 65, 2025, p. 103352.
- 24 Liu, R., D. Cui, X. Dai, P. Yue, and Z. Yuan, A Data-Driven Surrogate Modeling for Train
25 Rescheduling in High-Speed Railway Networks Under Wind-Caused Speed Restrictions.
26 *IEEE Transactions on Automation Science and Engineering*, Vol. 21, No. 2, 2024, pp.
27 1107–1121.
- 28 Sanchez-Gonzalez, A., J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. W. Battaglia, Learning to
29 Simulate Complex Physics with Graph Networks. In *Proceedings of the 37th International*
30 *Conference on Machine Learning (ICML)*, 2020, Vol. 119, pp. 8459–8468.
- 31 Chen, R. T. Q., Y. Rubanova, J. Bettencourt, and D. Duvenaud, Neural Ordinary Differential Equations.
32 In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018, pp. 6571–
33 6583.
- 34 Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polo-
35 sukhin, Attention Is All You Need. In *Advances in Neural Information Processing Systems*
36 *(NeurIPS)*, 2017, pp. 5998–6008.
- 37 Koenker, R. and G. Bassett, Regression Quantiles. *Econometrica*, Vol. 46, No. 1, 1978, pp. 33–50.
- 38 Romano, Y., E. Patterson, and E. J. Candès, Conformalized Quantile Regression. In *Advances in*
39 *Neural Information Processing Systems (NeurIPS)*, 2019, pp. 3538–3548.
- 40 OpenStreetMap contributors, *OpenStreetMap*. <https://www.openstreetmap.org>, 2026, accessed
41 July 2026.
- 42 U.S. Geological Survey, *3D Elevation Program (3DEP) Standards and Specifications*. U.S. Geo-
43 logical Survey, Reston, VA, 2022.
- 44 Savitzky, A. and M. J. E. Golay, Smoothing and Differentiation of Data by Simplified Least
45 Squares Procedures. *Analytical Chemistry*, Vol. 36, No. 8, 1964, pp. 1627–1639.